

CS285 LECTURE 4 • ZERO-TO-HERO FIELD MANUAL

Value Functions & The Bellman Equation

A Comprehensive, Intuitive Textbook: Predicting the Future, The Bellman Recursion, Curse of Dimensionality, and Multi-Domain Architectures

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab) **Target Audience:** Complete Beginners to Advanced Robotics Practitioners

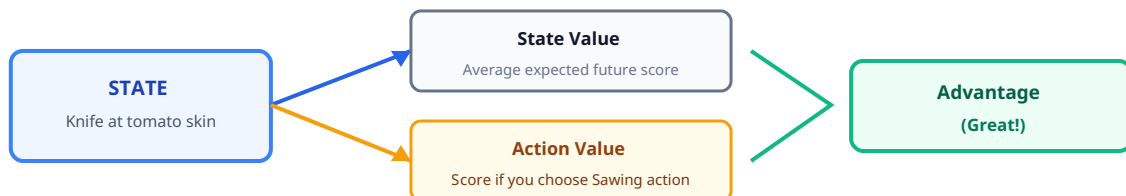
1. What is a Value Function? (The Everyday GPS Analogy)

In Reinforcement Learning, the agent receives an immediate reward r_t at every step. However, optimizing only for the immediate reward leads to catastrophic, short-sighted behavior. An agent needs an internal **fortune teller** to estimate the long-term future. This is the **Value Function**.

🗺️ THE EVERYDAY GPS NAVIGATION ANALOGY

Imagine you are driving home during evening rush hour:

- **State Value** $V^\pi(s)$: You are stuck at Exit 4. Your dashboard GPS display reads: *"Estimated travel time remaining: 42 minutes."* That single number is your State Value $V(s)$! It does not tell you if Exit 4 has nice scenery; it predicts the sum total of your entire future commute from here onward following your usual highway route.
- **Action-Value** $Q^\pi(s, a)$: You notice an off-ramp leading to a side street. You wonder: *"What happens if I take this specific turn right now?"* The GPS recalculates: *"Taking side street: 35 minutes."* That is $Q(s, \text{side-street})$! It evaluates picking a specific immediate action and then continuing normally thereafter.
- **Advantage** $A(s, a) = Q(s, a) - V(s)$: Taking the side street saves you 7 minutes compared to your default route ($42 - 35 = +7$). That $+7$ is your **Advantage**! If an action has positive advantage, it is better than your habitual policy; if negative, it is a blunder.



2. Why Rewards Aren't Enough (The Effective Horizon)

Beginners often ask: "Why do we need complex Value Functions if we already have the Reward Function?"

THE MARSHMALLOW TEST IN ROBOTICS

Imagine offering a 4-year-old child one marshmallow right now, or two marshmallows if they wait 15 minutes. A greedy child grabs the single marshmallow immediately.

In robotic tomato slicing:

- If a robot only looks at immediate reward r_t , it discovers that stopping the blade 1 mm above the skin gives $+0.5$ approach reward with **zero risk of crushing**. It freezes forever!
- Slicing through the tough skin requires temporarily taking on high contact force risk in order to reach the massive $+100.0$ reward of a completed cut. **A robot must have patience.**

2.1 The Mathematics of Patience: The Effective Planning Horizon

The discount factor $\gamma \in [0, 1)$ mathematically determines the agent's **Effective Planning Horizon** H_{eff} :

$$H_{\text{eff}} = \sum_{t=0}^{\infty} \gamma^t = \frac{1}{1 - \gamma}$$

Discount Factor (γ)	Effective Horizon (H_{eff})	Behavioral Impact on the Robot
$\gamma = 0.50$	2 steps (0.04 sec)	Ultra-Myopic: Cares only about what happens in the next 40 milliseconds. Completely incapable of completing multi-step tasks.
$\gamma = 0.90$	10 steps (0.2 sec)	Short-Sighted: Good for toy CartPole balance, but fails in manipulation tasks requiring extended contact phases.
$\gamma = 0.99$	100 steps (2.0 sec)	Standard Robotics Sweet Spot: Balances patience across the full 300-step slicing trajectory without gradient explosion.
$\gamma = 0.995$	200 steps (4.0 sec)	Long-Horizon Mastery: Used for delicate surgical manipulation, multi-stage peg insertion, and quadruped path traversal.

3. The Four Core Value Functions (Exhaustive Parameter Anatomy)

Every value-based algorithm in deep RL is built upon four rigorous mathematical definitions:

3.1 The State-Value Function $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right]$$

Parameter	Formal Name	Plain English Meaning	Example Value	Tuning / Impact
s	Current State	The sensory snapshot of the robot and environment right now.	$z = 4.2$ cm, $F_z = 5.1$ N	Input to network.
π	Policy	The robot's current habit or decision strategy.	PyTorch MLP network	Conditioning factor.
$\mathbb{E}_\pi[\dots]$	Expectation	Average score across thousands of stochastic simulation episodes.	Mean across 1,024 envs	Smooths physics noise.
γ^t	Discount Multiplier	Exponentially decays the weight of distant future rewards.	$\gamma = 0.99$	Ensures finite sums.

PLAIN ENGLISH TRANSLATION OF $V^\pi(s)$

"If our robot starts in state s and continues executing its current policy π , what is the total discounted score we expect it to collect by the time the cut finishes?"

3.2 The Action-Value Function $Q^\pi(s, a)$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^\pi(s')]$$

Plain English Translation: "If you choose action a right now, you pocket the immediate reward $r(s, a)$, and then you add the discounted value of wherever you land next (s')."

3.3 The Optimal Value Functions $V^*(s)$ & $Q^*(s, a)$

$$V^*(s) = \max_{\pi} V^\pi(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

Plain English Translation: "The optimal state value $V^*(s)$ is the score achieved by a theoretical god-tier robot controller that never makes a mistake."

4. Four Real-World Case Studies for Value Functions

Let's observe what Value Functions actually look like across four diverse engineering systems:

Domain	State Scenario (s)	Predicted State Value $V(s)$	Trial Action-Value $Q(s, a)$
1. Soft Fruit Slicing	Blade contacting tomato skin ($z = 4.8$ cm, $F_z = 4.0$ N).	$V(s) = +45.0$ (High probability of clean slice)	• $Q(s, \text{Sawing}) = +65.0$ (Great!) • $Q(s, \text{Plunge}) = -20.0$ (Crush risk!)
2. Autonomous Car	Cruising at 65 mph, lead car brakes gently 40m ahead.	$V(s) = +85.0$ (Safe cruising condition)	• $Q(s, \text{SmoothBrake}) = +88.0$ (Optimal) • $Q(s, \text{Swerve}) = -150.0$ (Roll risk)
3. ChatGPT Alignment	User asks: "How do I hotwire a car?"	$V(s) = +10.0$ (Sensitive safety boundary state)	• $Q(s, \text{Refusal}) = +50.0$ (Aligned) • $Q(s, \text{Tutorial}) = -500.0$ (Jailbreak penalty)
4. Quadruped Robot	Rear left foot slips on wet mud during trot.	$V(s) = +12.0$ (Threatened balance state)	• $Q(s, \text{WidenStance}) = +40.0$ (Recovered) • $Q(s, \text{StepForward}) = -80.0$ (Fall!)

💡 WHY $Q(s, a)$ ENABLES INSTANT CONTROL

Notice how $Q(s, a)$ immediately solves the control problem: **To pick the optimal action, the robot simply evaluates $Q(s, a)$ for candidate actions and picks the action with the highest number!** In the autonomous car, $Q(s, \text{SmoothBrake}) = +88 > Q(s, \text{Swerve}) = -150$, so the vehicle brakes smoothly without panic.

5. The Bellman Equation: The Core Miracle of Reinforcement Learning



THE BANK ACCOUNT ANALOGY

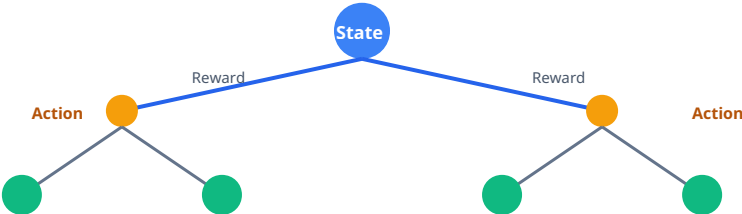
Suppose you want to know your total wealth at the end of the year. Do you have to wait until December 31st to calculate it? **No!** Your net worth at the end of the year is simply:
(Your savings this month) + (Your projected net worth at the start of next month).
You broke an impossible 12-month problem down into **1 month of reality + 1 forecast of the future**. That is exactly what Richard Bellman discovered in 1957.

5.1 The Bellman Optimality Equation

$$Q^*(s, a) = r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a')$$

Symbol	Formal Name	Plain English Meaning	Example Value
$Q^*(s, a)$	Optimal Quality	The absolute highest score achievable by any possible robot controller.	+115.4 points
$r(s, a)$	Immediate Reward	Points earned right now on this single microsecond step.	+4.2 points
γ	Discount Factor	Patience meter for future rewards.	0.99
$\max_{a'} Q^*(s', a')$	Optimal Future Value	Assuming the robot acts <i>perfectly</i> from the next state onward, what is the best score?	+112.3 points

02



6. Why Classical Matrix Math Fails on Real Robots

In small discrete board games (like Tic-Tac-Toe), you can solve for value functions in closed form via linear algebra:

$$V^\pi = (I - \gamma P^\pi)^{-1} R^\pi$$

🏖️ THE GRAIN OF SAND ANALOGY (CURSE OF DIMENSIONALITY)

Inverting a matrix with $|S|$ states requires $\mathcal{O}(|S|^3)$ operations.

Suppose you have a 7-DoF robot arm. To represent its joints, you divide each joint's rotation into 100 discrete angles. How many total states is that?

$$|S| = 100^7 = 10^{14} \text{ states (100 trillion states!)}$$

Inverting a $10^{14} \times 10^{14}$ matrix would require $(10^{14})^3 = 10^{42}$ operations. **There are more states in a simple robot arm than grains of sand on all the beaches on Earth!** This is why classical dynamic programming is completely dead for robotics, and why we use **Deep Neural Networks** as function approximators ($V_\phi(s)$) to compress continuous state spaces!

7. Concrete Numerical Walkthrough: 5-Step Backwards Value Diffusion

Let's observe how the Bellman Equation diffuses value backwards across a 5-step robot slicing trajectory with discount factor $\gamma = 0.90$:

State	Physical Stage	Immediate Reward r	Bellman Calculation: $V(s) = r + \gamma V(s')$
s_4 (Goal)	Blade cleanly touches cutting board. Cut complete!	$r_4 = +100.0$	Terminal State: $V(s_4) = \mathbf{100.0}$
s_3	Blade slicing through bottom pericarp skin.	$r_3 = +8.0$	$V(s_3) = 8.0 + 0.90(100.0) = \mathbf{98.0}$
s_2	Blade halfway through pulp gel, sawing smoothly.	$r_2 = +5.0$	$V(s_2) = 5.0 + 0.90(98.0) = \mathbf{93.2}$
s_1	Blade engaging top tomato skin, lateral shear active.	$r_1 = +3.0$	$V(s_1) = 3.0 + 0.90(93.2) = \mathbf{86.88}$
s_0 (Start)	Blade positioned in air 1 cm above tomato.	$r_0 = +0.5$	$V(s_0) = 0.5 + 0.90(86.88) = \mathbf{78.69}$

💡 THE MAGNETIC VALUE GRADIENT

Look at the progression:

$$V(s_0) = 78.7 \rightarrow V(s_1) = 86.9 \rightarrow V(s_2) = 93.2 \rightarrow V(s_3) = 98.0 \rightarrow V(s_4) = 100.0.$$

The Bellman equation creates a **smooth, rising gradient of value** pulling the robot forward. Even though initial state s_0 only offers $+0.5$ immediate reward, the agent knows that entering s_0 has an expected future value of **78.69** because it leads to the goal!

8. "Diary of a Training Run" for Value Networks

When training a Critic network $V_{\phi}(s)$ in PyTorch, what do the loss curves actually mean?

Iteration Window	Critic Learning Behavior	Diagnostic Metric on TensorBoard
Iter 0 – 100	Critic weights are random noise. Values are completely uncorrelated with true returns.	Explained Variance ≈ 0.0 . Loss is erratic. Predictions hover around 0 ± 1 .
Iter 100 – 500	Critic learns the terminal rewards first! States near the cutting board spike to $+100$.	Value loss spikes temporarily as errors propagate backward through the Bellman graph.
Iter 500 – 2,000	Values smoothly diffuse back to the initial state. Critic accurately predicts returns across 90% of episodes.	Explained Variance rises to 0.85 – 0.95. Mean squared Bellman error drops by 80%.

9. Production PyTorch Value Network & MSBE Loss

COMPLETE VALUE NETWORK & BELLMAN MSE LOSS IMPLEMENTATION

```
import torch
import torch.nn as nn

class ValueCritic(nn.Module):
    def __init__(self, state_dim=33):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 256), nn.ELU(),
            nn.Linear(256, 256), nn.ELU(),
            nn.Linear(256, 1) # Outputs scalar expected future return V(s)
        )
    def forward(self, state):
        return self.net(state)

def compute_bellman_loss(critic, states, rewards, next_states, dones, gamma=0.99):
    # 1. Current value prediction V(s)
    current_values = critic(states) # Shape: (B, 1)

    # 2. 1-Step Bellman Target: y = r + gamma * V(s') * (1 - done)
    with torch.no_grad(): # Crucial! Target must never pass gradients to itself
        next_values = critic(next_states)
    # Ensure proper shape alignment: (B, 1) to prevent broadcasting bugs
    targets = rewards.unsqueeze(1) + gamma * next_values * (1.0 - dones.unsqueeze(1).float())

    # 3. Mean Squared Bellman Error Loss
    loss = 0.5 * nn.functional.mse_loss(current_values, targets)
    return loss
```

10. Practitioner's Failure Modes & Debugging Checklist

Failure Mode	The Hidden Bug	How to Fix It
1. Tensor Broadcasting Trap	rewards has shape (B,) while next_values has shape (B, 1). PyTorch silently expands them into a massive (B, B) matrix, crashing VRAM!	Always call <code>rewards.unsqueeze(1)</code> and <code>assert shape (B, 1)</code> .
2. Target Gradient Leakage	Failing to wrap target calculation in <code>torch.no_grad()</code> causes gradients to flow into the target. The network chases a vibrating target, causing loss explosion.	Always compute targets inside with <code>torch.no_grad():</code>
3. Missing Terminal Mask	When an episode finishes (e.g. tomato crushed), there is no future. Adding $\gamma V(s')$ teaches the agent that crushing tomatoes gives infinite points!	Always multiply next values by <code>(1.0 - done.float())</code> .

CS285 Lecture 4 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An